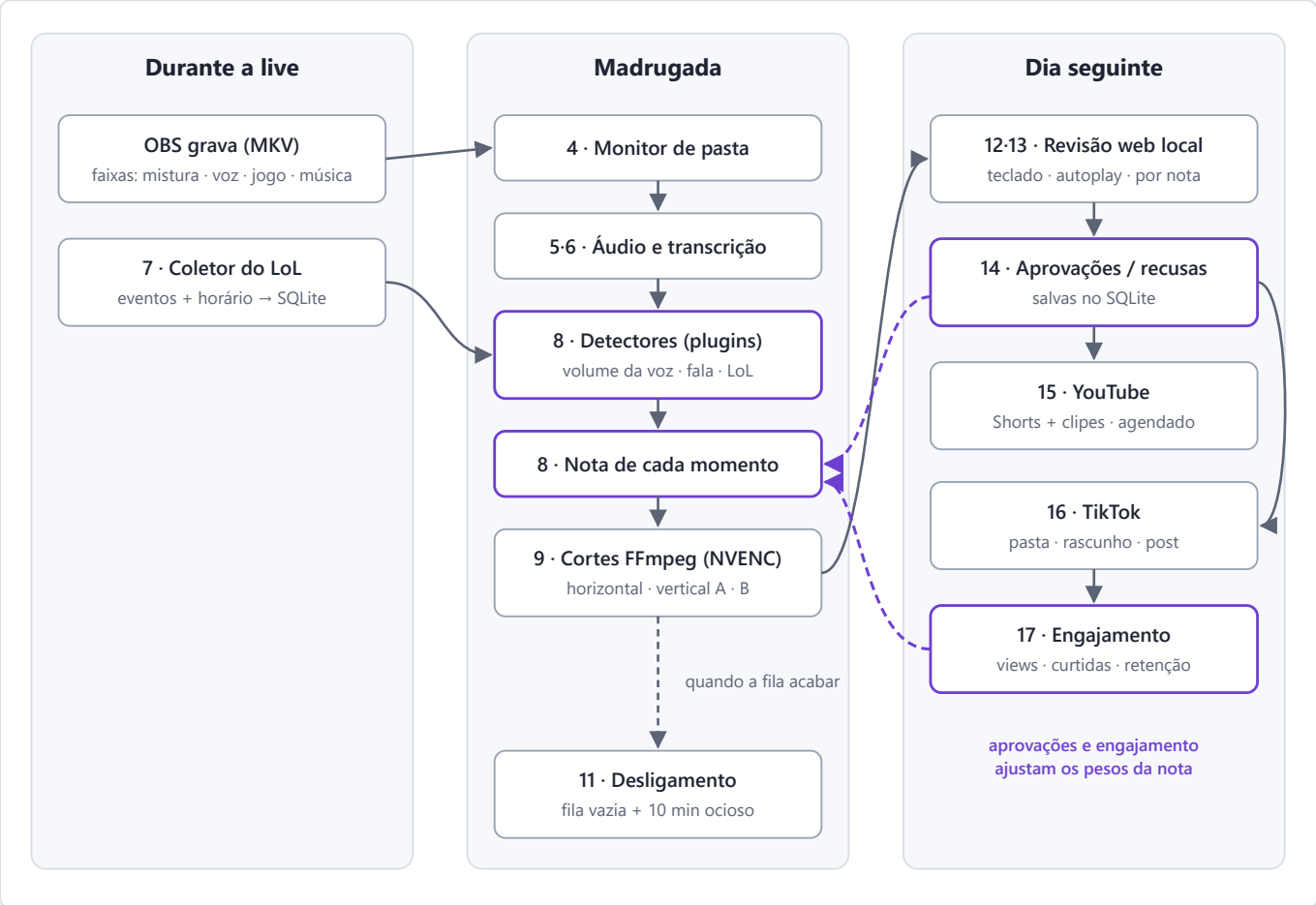
Clipador de lives

Stack, ferramentas, roteiro completo e riscos · nome provisório · outubro de 2026

100% gratuito | Python 3.12 | GTX 1050 Ti · 4 GB | Windows 10 | Projeto de portfólio

App pessoal que transforma as gravações diárias da live (OBS, ~3h) em clipes para YouTube e TikTok. O processamento roda de madrugada, a revisão acontece no dia seguinte numa interface web local e **nada é publicado sem aprovação**.

1. Pipeline



Os números são as etapas do roteiro (seção 3). As etapas 0 a 3 (base), 10 (títulos, que roda junto dos cortes), 18 (vitrine) e 19 (futuro) não aparecem no diagrama. Bordas roxas: o "ciclo de aprendizado" do projeto. As linhas tracejadas roxas são o diferencial de portfólio: as suas decisões e o engajamento do público viram dados para melhorar a nota.

IDEIAS DE ARQUITETURA

- **Detectores como plugins**: interface comum `Detector.detectar(entrada) -> list[Sinal]` (trecho, força de 0 a 1 e motivo); um passo separado junta os sinais em momentos e dá a nota (ADR 0005). Outro jogo ou o chat da Twitch vira só um plugin novo, sem depender de um jogo só.
- **Fila retomável em SQLite**: cada etapa salva o próprio estado. Se o PC desligar ou travar, o processamento continua de onde parou.
- **Aprovações como dados**: medir a precisão do detector ("dos 15 sugeridos, aprovei 9") e depois ajustar os pesos com uma regressão logística.

- **Engajamento como dados**: visualizações, curtidas e retenção dos cortes publicados ensinam a nota a prever o que o público gosta, não só o que você aprova.
- **Vitrine técnica**: CI no GitHub Actions, ADRs curtos em `docs/adr/`, GIF da interface e diagrama da pipeline no README, modo demo com dados de exemplo.

2. Ferramentas

Todas são gratuitas. A maioria também é de código aberto; as exceções são serviços e componentes de fabricantes (GitHub, drivers e CUDA da NVIDIA, comandos do Windows). As APIs do YouTube e do TikTok são gratuitas, mas têm limites (veja a seção 4).

BASE DO PROJETO

FERRAMENTA	POR QUE USAR
Python 3.12	Melhor ecossistema para áudio, vídeo e IA, e já está instalado no PC.
uv	Gerencia Python, ambiente virtual e dependências num comando só, com lockfile. Qualquer pessoa clona o repositório e roda igual.
ruff	Lint e formatação numa ferramenta só, muito rápida. Mantém o código com cara de projeto profissional.
pytest	Testes dos detectores e do alinhamento de tempo, que são as partes mais fáceis de quebrar sem perceber.
pytest-cov	Mede quanto do código os testes cobrem. Gera o número do selo de cobertura no README.
pre-commit	Roda ruff e testes rápidos antes de cada commit, para nada quebrado chegar no GitHub.
pydantic-settings + TOML config.toml / .env fora do git	Configuração validada e tipada: um valor errado gera um erro claro no início, não no meio da madrugada. As chaves ficam fora do código.
Typer	Cria a linha de comando do app (`clipador status` , `clipador demo` ...) a partir de funções Python com tipos, com ajuda automática.
logging (biblioteca padrão)	Log em arquivo rotativo, sem dependência extra. Essencial para saber o que aconteceu enquanto você dormia.

PROCESSAMENTO

FERRAMENTA	POR QUE USAR
watchdog	Avisa quando aparece uma gravação nova na pasta. O app espera o arquivo parar de crescer antes de processar.
FFmpeg build com NVENC (ex.: gyan.dev "full")	Padrão da indústria para vídeo: separa faixas, corta, monta os verticais e remove a música. O encoder `h264_nvenc` usaria o chip de vídeo da 1050 Ti, mas o FFmpeg 9 exige um driver (610+) que a placa não recebe; os cortes saem no processador (libx264), ~14 s para 20 s de vertical em 1080p (ADR 0006, conferido na etapa 9).
faster-whisper modelo medium · int8 · CUDA	Mesmo modelo do Whisper, até ~4x mais rápido e com menos memória. Em int8 o `medium` cabe nos 4 GB. Dá timestamps por palavra, usados no alinhamento e nas legendas. O filtro de silêncio (VAD) evita que ele "invente" frases em trechos sem fala.

CUDA 12 + cuDNN 9 bibliotecas da NVIDIA	Necessárias para o faster-whisper usar a placa. Instaláveis pelo próprio pip (pacotes `nvidia-cublas-cu12` e `nvidia-cudnn-cu12`), sem instalar o CUDA Toolkit inteiro. No Windows, o app pode precisar apontar para a pasta dessas DLLs na inicialização. Precisa ser a versão 12: veja o risco "GTX 1050 Ti" na seção 4.
numpy + soundfile	Calcula o volume da faixa do microfone ao longo do tempo para achar os picos da sua voz, o sinal principal do detector.
httpx	Cliente HTTP moderno. Usado pelo coletor da Live Client Data API do LoL (`https://127.0.0.1:2999`, com certificado próprio da Riot) e pela API do TikTok.
Ollama + qwen3:4b IA de texto local	Roda um modelo de IA no próprio PC, gratuito e sem internet, para sugerir títulos e hashtags com cara de gente (etapa 10). O qwen3:4b ocupa 2,5 GB e cabe na 1050 Ti.
SQLite + SQLModel	Banco num arquivo só, sem servidor. Guarda fila, momentos, eventos do LoL e aprovações. O SQLModel junta tabelas e validação com tipos.
Alembic	Migrações do banco: quando uma tabela muda numa etapa futura, o banco que já existe é atualizado sem perder dados.
scikit-learn	Regressão logística para ajustar os pesos da nota com base nas suas aprovações e recusas, e métricas de precisão para o README.

INTERFACE DE REVISÃO

FERRAMENTA	POR QUE USAR
FastAPI + Uvicorn	API em Python com tipos e documentação automática em `/docs`. Serve os clipes e grava as aprovações.
Node.js LTS + npm	Necessário para instalar e rodar as ferramentas do frontend (Vite, React, Tailwind). Só é usado no desenvolvimento: o build final vira arquivos estáticos servidos pelo FastAPI.
React + TypeScript	Frontend mais pedido em vagas. TypeScript evita erros bobos e combina com os tipos do backend.
Vite	Servidor de desenvolvimento e build do frontend, rápido e simples de configurar.
Tailwind CSS	Interface bonita e consistente sem escrever muito CSS.
Recharts	Gráficos em React para a tela de Estatísticas (precisão do detector ao longo do tempo).
Vitest	Testes do frontend com a mesma configuração do Vite.

PUBLICAÇÃO

FERRAMENTA	POR QUE USAR

YouTube Data API v3 google-api-python-client + google-auth-oauthlib	Upload e agendamento dos Shorts e dos clipes horizontais. Biblioteca oficial do Google, com login OAuth. Não existe API separada para Shorts: um vídeo vertical de até 3 minutos é classificado como Short automaticamente.
YouTube Analytics API	Gratuita. Quanto de cada vídeo as pessoas assistem (retenção), usado na etapa 17. Na etapa 19, também os melhores horários do seu público.
TikTok Display API	Lê visualizações, curtidas, comentários e compartilhamentos dos seus vídeos no TikTok (etapa 17).
TikTok Content Posting API	Envio como rascunho e, depois da auditoria, post direto. Exige criar um app no TikTok for Developers; dá para testar no modo sandbox com a sua conta. Enquanto isso, a opção "salvar numa pasta" funciona sem API.

SISTEMA E DESLIGAMENTO

FERRAMENTA	POR QUE USAR
ctypes + SetThreadExecutionState	Função nativa do Windows que impede o PC de entrar em suspensão no meio do processamento da madrugada. Sem instalar nada.
ctypes + GetLastInputInfo	Função nativa do Windows que diz há quanto tempo o teclado e o mouse estão parados. Usada na regra dos 10 minutos.
Chave "Run" do registro (HKCU)	Inicia o app sozinho quando você faz login, sem precisar de administrador (o Agendador de Tarefas exige administrador para isso; ADR 0004). Assim o coletor do LoL já está rodando quando a partida começa (ele consulta a API a cada poucos segundos e só grava quando há partida) e o monitor de pasta pega a gravação assim que a live termina.
shutdown.exe	Comando do próprio Windows: `shutdown /s /t 60` desliga em 60 s e `shutdown /a` cancela.

REPOSITÓRIO E VITRINE

FERRAMENTA	POR QUE USAR
Git + GitHub	Histórico do projeto e vitrine pública para recrutadores.
GitHub Actions	CI que roda lint e testes a cada push. Grátis para repositórios públicos e gera o selo "passing" no README.
Mermaid	Diagramas escritos em texto dentro do README. O GitHub desenha sozinho.
ADRs (Markdown)	Um arquivo curto por decisão ("por que SQLite e não Postgres"). Mostra raciocínio de engenharia.

FUTURO (ETAPA 19)

FERRAMENTA	POR QUE USAR
twitchio	Lê o chat da Twitch durante a live. Picos de mensagens viram mais um sinal (um plugin novo).

MediaPipe	Detecta rosto e expressão na câmera para usar sua reação como sinal. Roda localmente.

3. Roteiro do projeto, etapa por etapa

As etapas seguem a **ordem de estrutura**: cada uma só depende das anteriores. Primeiro vem a base, que todo o resto usa (ambiente, configuração, banco de dados, dados de exemplo). Depois vem a pipeline na ordem em que os dados passam por ela, e por fim a revisão, a publicação e a vitrine.

Toda etapa segue o mesmo ciclo: **plano** → **código** → **testes** → **parar e esperar sua aprovação**. Cada etapa termina com um commit, a CI verde no GitHub e o README atualizado com o que o app passou a fazer. Em cada uma, "Instalar" lista o que entra de novo e "Pronto quando" é o critério para passar para a próxima.

Fase A · Fundação a base que todas as outras etapas usam

Etapa 0 Preparação do PC

Objetivo	Deixar o PC pronto para desenvolver e o OBS gravando no formato que o app espera. Nesta etapa ainda não tem código.
Instalar	• **Primeiro, resolver o bloqueio do pip** (erro "certificate verify failed: self-signed certificate"). Quase sempre é um antivírus (Avast, Kaspersky, ESET etc.) com a "verificação de HTTPS/SSL" ligada, ou um proxy. Desligue essa verificação no antivírus ou abra uma exceção para o Python. Não desative a verificação de certificado do pip. Sem isso, nenhuma biblioteca instala. • **Driver da NVIDIA** da série **570 a 580**. Abaixo de 570 o Ollama (etapa 10) não usa a placa; a série 580 é a última que suporta placas Pascal como a 1050 Ti (*conferido*; veja o risco "GTX 1050 Ti" na seção 4). Confira a versão com `nvidia-smi`. • **uv** (o Python 3.12 já está instalado): `winget install astral-sh.uv` • **FFmpeg** com NVENC: `winget install Gyan.FFmpeg` • **Node.js LTS**: `winget install OpenJS.NodeJS.LTS` • **Git**: `winget install Git.Git`. Depois configure nome e e-mail (`git config --global user.name` e `user.email`) e crie uma conta no GitHub. • **VS Code** (opcional), com as extensões Python e Ruff. • Depois dos `winget install`, **feche e abra o terminal** para os comandos novos serem encontrados.

Configurar o OBS	• **Configurações → Saída → modo de saída "Avançado" → aba Gravação**: formato **MKV** e faixas de áudio **1, 2, 3 e 4** marcadas. • **Pasta de gravação** num disco com bastante espaço livre (100 GB ou mais; veja o risco "Espaço em disco"). • **Fontes separadas**: o "Áudio da área de trabalho" mistura tudo o que toca no PC. Use **Captura de áudio de aplicativo** (OBS 28 ou mais novo, Windows 10 versão 2004 ou mais nova): uma fonte para o LoL e outra para o player de música. Desative o "Áudio da área de trabalho" para nada sair duplicado. • **Edição → Propriedades avançadas de áudio**, em "Faixas": ◦ **Faixa 1 = mistura completa** (voz + jogo + música): marcar nas três fontes. É a faixa que vai para a Twitch. ◦ **Faixa 2 = voz**: só no microfone. ◦ **Faixa 3 = jogo**: só na captura do LoL. ◦ **Faixa 4 = música**: só na captura do player. • Na aba **Transmissão**, a "Faixa de áudio" continua sendo a **1**. Se a voz ficasse sozinha na faixa 1, a live iria ao ar só com a sua voz. • Mantenha o nome de arquivo padrão do OBS, com data e hora (ex.: `2026-10-14 20-00-00.mkv`). O app usa esse horário no alinhamento com o LoL.
Pronto quando	• `python --version` (3.12), `uv --version`, `node --version`, `git --version` e `nvidia-smi` funcionam num terminal novo, e o `nvidia-smi` mostra driver entre 570 e 580. • `ffmpeg -hide_banner -encoders | findstr nvenc` mostra `h264_nvenc`. Atenção: aparecer na lista não garante que funcione; com o FFmpeg 9 e o driver 580, ele falha ao abrir (etapa 9, ADR 0006), e o app corta no processador. • `python -m pip install --dry-run requests` roda sem erro de certificado. • Uma gravação de teste de 1 minuto, conferida com `ffprobe`, tem **4 faixas de áudio**. A faixa 1 tem o som completo, e a música não aparece nas faixas 2 e 3.

Etapa 1 Esqueleto do projeto

Objetivo	Criar a estrutura do repositório e as ferramentas de qualidade antes de qualquer funcionalidade. Assim, todo código novo já nasce com lint, testes e CI.
Instalar	O projeto é criado com `uv init --package clipador` (estrutura `src/`) e o Python fixado em 3.12 (`.python-version`). Depois, via `uv add`: pydantic-settings e Typer. Ferramentas de desenvolvimento (`uv add --dev`): ruff, pytest e pre-commit.

Código	Estrutura de pastas em **quatro partes**, com uma direção de dependência só (ADR 0003): `frontend ──(HTTP)──> api ──> backend ──> integracoes ──> YouTube / TikTok / LoL`. Cada pasta é preenchida na etapa indicada:
	
	`Clipador/`
	`├─ integracoes/ único código que lê as chaves (.env)`
	`│ └─ src/clipador_integracoes/`
	`│ ├─ lol.py cliente da Live Client Data API (etapa 7)`
	`│ ├─ youtube.py upload e métricas do YouTube (etapas 15 e 17)`
	`│ └─ tiktok.py rascunho, post e métricas (etapas 16 e 17)`
	`├─ backend/ usa as integrações; nunca lê as chaves`
	`│ ├─ src/clipador/`
	`│ │ ├─ config.py · logs.py · cli.py (etapa 1)`
	`│ │ ├─ banco/ modelos e fila (etapa 2)`
	`│ │ ├─ demo/ · ffmpeg.py (etapa 3)`
	`│ │ ├─ sistema/ serviço, energia (etapas 4 e 11)`
	`│ │ ├─ ingestao/ monitor e áudio (etapas 4 e 5)`
	`│ │ ├─ transcricao/ faster-whisper (etapa 6)`
	`│ │ ├─ jogo/ coleta e alinhamento (etapa 7)`
	`│ │ ├─ detectores/ plugins e nota (etapa 8)`
	`│ │ ├─ cortes/ FFmpeg (etapa 9)`
	`│ │ ├─ titulos/ IA local (Ollama) (etapa 10)`
	`│ │ ├─ aprendizado/ métricas e pesos (etapas 14 e 17)`
	`│ │ └─ publicacao/ tarefas de publicar (etapas 15 a 17)`
	`│ └─ tests/`
	`├─ api/ usa o backend; nunca as integrações direto`
	`│ └─ src/clipador_api/ FastAPI (etapa 12)`
	`├─ frontend/ React + Vite (etapa 13); nunca guarda chaves`
	`├─ tests/ testes de arquitetura (regras entre as partes)`
	`├─ dados/ fora do git: banco, logs, clipes, demo`
	`├─ docs/adr/`
	`├─ .github/workflows/ci.yml`
	`├─ config.example.toml · .env.example`
	`├─ pyproject.toml workspace do uv (liga as 3 partes Python)`
	`└─ README.md · LICENSE · .pre-commit-config.yaml`
	
	• **Workspace do uv**: `integracoes/`, `backend/` e `api/` têm cada uma o próprio `pyproject.toml` e as próprias dependências, mas são instaladas juntas (`uv sync`) num único ambiente, com um único `uv.lock`. O frontend fica fora, com o próprio `package.json`.
	• **Teste de arquitetura** (`tests/test_arquitetura.py`): lê o código de cada parte e falha se alguém quebrar as regras (ex.: a API importar as integrações, ou o backend ler o `.env`).
	• `config.py`: lê o `config.toml` (pastas, faixas do OBS, pesos, limite diário, desligamento e o que as próximas etapas acrescentarem) e variáveis de ambiente `CLIPADOR_*`, e valida tudo com pydantic. **Não lê o `.env`**: ele guarda as chaves, e só as integrações o leem.
	• `logs.py`: log em arquivo rotativo em `dados/logs/`.
	• `cli.py`: comando `clipador` com o subcomando `clipador versao`.
	• `.gitignore` com `config.toml`, `.env`, `dados/` e os arquivos de vídeo.
	• CI: GitHub Actions roda ruff e pytest a cada push, em **Windows** (`windows-latest`), porque o app usa funções do Windows.
	• Primeiro ADR: "Por que Python + SQLite + FastAPI".
	• `LICENSE` (MIT) desde o início, para o repositório já nascer público e reutilizável.

Funcionalidade	Nenhuma funcionalidade de clipes ainda. O comando `clipador versao` roda, e uma configuração inválida gera um erro claro.

Testes	A configuração de exemplo carrega sem erro. Um valor inválido (ex.: limite diário negativo) é rejeitado. `clipador versao` responde (teste da CLI com o `CliRunner` do Typer).
Pronto quando	Repositório no GitHub com CI verde, pre-commit funcionando e README inicial explicando o objetivo do projeto.

Etapa 2 Banco de dados e fila retomável

Objetivo	Criar o "estado" do app. Todas as etapas seguintes leem e gravam aqui, por isso ele vem antes de qualquer processamento.
Instalar	SQLModel e Alembic (o SQLite já vem com o Python).

Código	• `banco/modelos.py`, com as tabelas: ◦ `Gravacao`: arquivo, início, duração, status. ◦ `Tarefa`: tipo, gravação, momento (para a tarefa `recortar`) ou publicação a que se refere, status (pendente, em andamento, feita, erro), tentativas, último erro, **executar a partir de** (para tarefas agendadas, como a coleta de métricas e o reenvio depois que a cota zerar). ◦ `Transcricao`: gravação, palavras com início e fim. ◦ `EventoJogo`: tipo, horário real, tempo de jogo, dados do evento. ◦ `Momento`: gravação, início, fim, sinais, nota, motivo, títulos e hashtags sugeridos (etapa 10). ◦ `Clipe`: momento, formato, arquivo, duração. ◦ `Decisao`: momento, aprovado/recusado, formatos e destinos escolhidos, título e hashtags escolhidos, tipo do título (etapa 10), ajuste do corte (início e fim), data. ◦ `Publicacao`: clipe, plataforma, **ID do vídeo na plataforma** (necessário para buscar métricas), status, link, agendada para, unidades de cota gastas, data. ◦ `PesosNota`: versão dos pesos, valores, data, precisão medida (etapas 14 e 17). ◦ `Metrica`: publicação, momento da coleta (24 h, 72 h, 7 dias), visualizações, curtidas, comentários, compartilhamentos, retenção (etapa 17). • **Tipos de tarefa**. A cadeia principal é `extrair_audio → transcrever → detectar → cortar → gerar_titulos`; quando uma termina, ela cria a próxima. Outras tarefas, criadas por eventos: ◦ `recortar`: quando você ajusta um corte (etapa 12). ◦ `publicar`: só depois da sua aprovação (etapas 15 e 16). ◦ `coletar_metricas`: agendada para 24 h, 72 h e 7 dias após a publicação (etapa 17). • `banco/fila.py`: pega a próxima tarefa pendente e a marca como "em andamento". Se o app cair, tarefas "em andamento" voltam para "pendente" na próxima abertura. Uma tarefa que falha é tentada de novo até 3 vezes; depois vira "erro" com a mensagem guardada. • **SQLite em modo WAL**: o serviço da madrugada e a interface web acessam o banco ao mesmo tempo sem travar um ao outro. • **Alembic** para migrações: quando uma etapa futura muda uma tabela, o banco existente é atualizado sem perder dados. A migração roda com as chaves estrangeiras desligadas (o SQLite recria a tabela, e isso falharia com elas ligadas), confere tudo com `PRAGMA foreign_key_check` e, num banco que já existia, faz um backup antes. • **Backup do banco** (`banco/backup.py`): o banco é o "caderno" do app, com aprovações, métricas, hashtags, títulos e pesos aprendidos. Ele é pequeno, mas não dá para recriar. O app faz uma cópia por dia usando a função de backup do próprio SQLite, que é segura mesmo com o banco em uso, e guarda as últimas 14 numa pasta do config. O ideal é outro disco ou uma pasta sincronizada com o Google Drive. Comando manual: `clipador backup`. • CLI: `clipador status` mostra a fila e os erros.
Funcionalidade	O app guarda o que precisa fazer e o que já fez. Desligar no meio não perde trabalho.

Testes	Em banco temporário: uma tarefa interrompida volta para a fila. Uma tarefa que falha 3 vezes vira "erro" e não trava as outras. Terminar uma tarefa cria a próxima da cadeia. Uma tarefa agendada não roda antes da hora. A migração inicial cria todas as tabelas. O backup gera uma cópia que abre e tem os mesmos dados, e só as últimas 14 ficam guardadas.
Pronto quando	`clipador status` mostra uma fila de teste. ADR: "Por que SQLite e não Postgres".

Etapa 3 Modo demo e dados de exemplo

Objetivo	Ter uma "live falsa" pequena para testar todas as etapas seguintes sem depender de uma gravação de 3 horas. Ela também vira o modo demo da entrevista.
Instalar	Nada novo. Usa o FFmpeg.
Código	• `demo/gerar.py`: usa o FFmpeg para criar um MKV de 3 minutos com imagem de teste e as mesmas 4 faixas do OBS, feitas de sons sintéticos (ruído e tons). A faixa de voz tem picos de volume em horários conhecidos. • Junto com o vídeo: eventos falsos do LoL, uma transcrição pronta e títulos prontos. Por isso a demo **pula o Whisper e a IA de títulos e não precisa de GPU**: roda em qualquer notebook, inclusive o da entrevista. • CLI: `clipador demo` usa uma pasta e um banco separados (`dados/demo/`) e roda tudo neles, sem tocar nos dados reais. `--do-zero` apaga e recria; `clipador status --demo` mostra a fila da demo. • **Escolhas medidas para ficar rápida** (cerca de 15 s): o vídeo é um fundo liso com o tempo da gravação escrito no centro (o padrão de teste `testsrc` levava 28 s, e assim se vê em que segundo está cada pico), e o áudio sai em Opus (o AAC levava 24 s para as 4 faixas). Uma gravação real do OBS vem em AAC, mas o FFmpeg lê os dois. • A CI instala o FFmpeg pelo Chocolatey (`ffmpeg-full` , a mesma build do PC; a imagem Windows do GitHub não traz o FFmpeg). A demo real de 3 minutos é um teste marcado como `lento`: roda na CI, não no pre-commit.
Funcionalidade	Qualquer pessoa clona o repositório, roda `clipador demo` e vê o app funcionando. Basta ter Python (via uv) e FFmpeg; não precisa de GPU, Ollama nem contas no YouTube ou TikTok.
Testes	A gravação demo tem 4 faixas e a duração certa. Os picos ficam nos horários esperados. Rodar a demo duas vezes não duplica nada.
Pronto quando	O comando gera os arquivos de exemplo em menos de 1 minuto. Nas próximas etapas, cada recurso novo ganha o seu pedaço na demo; nas etapas 14 e 17, por exemplo, entram decisões e métricas de engajamento falsas para os gráficos.

Fase B · Pipeline noturna na ordem em que a gravação passa

Etapa 4 Monitor de pasta e execução em segundo plano

Objetivo	Perceber sozinho que uma live terminou, colocar a gravação na fila e processar sem atrapalhar a live.
Instalar	watchdog.

Código	• `ingestao/monitor.py`: observa a pasta do OBS. Um arquivo novo só entra na fila depois de ficar 2 minutos sem mudar de tamanho, ou seja, quando a gravação terminou. • Ao abrir, o app também **varre a pasta** e pega gravações que ainda não estão no banco. Assim nada se perde se ele estava fechado quando a live acabou. • `sistema/servico.py`: laço principal. O monitor e o coletor do LoL (etapa 7) rodam em paralelo, em threads, e as tarefas da fila são executadas uma por vez. • **Enquanto houver uma gravação em andamento** (arquivo ainda crescendo), a fila fica parada, para o processamento não disputar CPU e GPU com a live. • O serviço também roda o **backup diário do banco** (etapa 2), uma vez por dia, e executa as **tarefas agendadas** quando chega a hora delas (coleta de métricas, reenvio depois que a cota zerar). Se o PC estava desligado na hora marcada, a tarefa roda assim que ele ligar. • `sistema/energia.py`: `SetThreadExecutionState` impede a suspensão do Windows enquanto houver fila e libera quando ela acaba. • CLI: `clipador iniciar`, mais `clipador instalar-servico`, que cadastra o app para abrir no login pela **chave "Run" do registro do seu usuário** (o Agendador de Tarefas exigiria administrador; ADR 0004), sem janela (`pythonw`), com tudo indo para o log. • **Feito em 2026-10-15, com estes detalhes:** na primeira vez, o monitor usa a data de modificação do arquivo como ponto de partida (uma gravação antiga já entra na varredura ao abrir); se o OBS deixar a mesma live em .mkv e .mp4 (remux), só o .mkv entra; a pausa durante a live fica registrada no log; o serviço só pega tarefas que já sabe executar (as outras esperam a etapa delas, em vez de falhar à toa); uma trava garante um serviço só por vez; o serviço escreve em `servico.log` (no Windows, dois processos no mesmo log travam a rotação); sem janela, um erro de configuração vai para `clipador-servico-erro.txt`. `clipador remover-servico` desfaz a abertura no login. Por segurança, só entram vídeos com o nome no padrão do OBS (o OBS grava em Vídeos por padrão, junto com outros vídeos); `so_nomes_do_obs = false` desliga isso. O modo HLS do OBS (vários pedaços `.ts`) não é suportado. Os clipes do Buffer de Replay (nomes que começam com "Replay") são ignorados. Com a opção "dividir gravação automaticamente" do OBS, cada pedaço vira uma gravação separada. O tempo de 2 minutos é configurável (`minutos_gravacao_parada`): quem usa o "Pausar gravação" do OBS por mais tempo que isso precisa aumentá-lo, senão a live entra na fila pela metade. O `clipador status` avisa se o cadastro no login aponta para outro `config.toml` (pasta do projeto movida). Uma tarefa que já começou não é interrompida pelo serviço se uma live começar: as tarefas longas (etapas 6 e 9) trabalham em blocos e, entre um e outro, perguntam `contexto.deve_pausar()`; se for sim, salvam o que fizeram e levantam `fila.TarefaPausada`, que devolve a tarefa à fila sem contar tentativa (feito na etapa 6).
Funcionalidade	Terminou a live, a gravação entra na fila sozinha e o processamento começa. O PC não dorme no meio.
Testes	Um arquivo ainda crescendo não entra na fila e mantém a fila parada. Um arquivo estável entra uma vez só. Arquivos que não são vídeo são ignorados. A varredura na abertura encontra gravações antigas não processadas. Uma tarefa agendada atrasada roda assim que o serviço abre.
Pronto quando	Gravar 1 minuto no OBS e a gravação aparecer em `clipador status` sem fazer nada. Depois de reiniciar o PC, o app abre sozinho.

Etapa 5 **Extração do áudio da voz**	
Objetivo	Tirar da gravação o áudio da sua voz, no formato que o Whisper e o detector de volume usam, e registrar os dados do vídeo.
Instalar	soundfile e numpy.
Código	• `ingestao/audio.py` : usa o `ffprobe` para conferir faixas, duração, resolução e fps. Os números das faixas vêm do `config.toml` ; para o FFmpeg, a faixa 2 do OBS é o segundo fluxo de áudio (`0:a:1`). • Extrai a voz em WAV mono 16 kHz (o formato que o Whisper espera). As faixas de jogo e música não são extraídas: os cortes da etapa 9 leem direto do MKV. • Lê o horário de início pelo nome do arquivo. Se o nome não seguir o padrão do OBS, usa a data de criação do arquivo e avisa no log. • Se faltar faixa, a tarefa falha com uma mensagem clara ("gravação sem a faixa 2 (voz): confira o OBS"). • Os arquivos temporários em `dados/trabalho/` são apagados quando a gravação termina a pipeline. • **Feito em 2026-10-16, com estes detalhes:** as faixas ficam em `[faixas]` no `config.toml` (voz, jogo e música, contadas a partir de 1 como no OBS; precisam ser diferentes). O WAV fica em `dados/trabalho/<id>/voz.wav` e é gravado com outro nome e só renomeado no fim, para um arquivo pela metade nunca parecer pronto. Dois erros comuns de configuração do OBS viram **falha definitiva**, que vai direto para "erro" em vez de ser tentada 3 vezes: gravação sem a faixa da voz e faixa da voz em silêncio (microfone não marcado na faixa 2). Faltar a faixa do jogo ou da música só gera aviso no log. Também confere o espaço em disco antes (uma live de 3 h dá cerca de 350 MB de WAV) e avisa se a duração do WAV e a do vídeo diferirem mais de 2 s. A gravação ganhou largura, altura e fps (migração 0002) e passa a ter situação: "processando" na extração, "concluída" no fim da cadeia (quando os temporários são apagados) e "erro" quando uma tarefa da cadeia desiste; se der erro, o WAV fica, para investigar. O horário de início pelo nome já era lido na etapa 4. Uma gravação interrompida (OBS travou, faltou luz) fica sem a duração no cabeçalho do MKV, que só é escrita quando a gravação termina normalmente; ela é aproveitada mesmo assim, com a duração medida no próprio WAV (simulado matando o FFmpeg no meio da gravação). Na CI, o FFmpeg instalado pelo Chocolatey é um "shim" (um programa que abre o FFmpeg verdadeiro): o limite de tempo encerrava só o shim e a CI travou por 5 h; agora o processo é encerrado com os filhos (`taskkill /T`), e cada teste tem limite de 5 min. Depois de corrigir a causa de um erro, `clipador refazer <número>` devolve a tarefa para a fila (o `status` lembra disso). O `ffprobe` também ganhou limite de tempo (60 s), para um arquivo corrompido não travar a fila. Novo comando `clipador processar [--demo]` : processa a fila na hora, sem o serviço (na demo, sem vigiar a pasta, porque o vídeo recém-criado pareceria uma live em andamento). Testado em 5 gravações reais do OBS (MP4/AAC, de 1,5 a 25 min): diferença de no máximo 0,06 s entre o WAV e o vídeo, e cerca de 100 vezes mais rápido que o tempo real (uma live de 3 h leva uns 2 min).
Funcionalidade	Cada gravação gera o áudio da voz pronto para a transcrição e para o detector de volume.

Testes	Na gravação demo: a duração do WAV bate com a do vídeo e os picos estão nos lugares certos. Um nome de arquivo fora do padrão ainda gera um horário de início. Uma gravação sem a faixa de voz dá erro claro.
Pronto quando	A extração funciona na demo e numa gravação real do OBS.

Etapa 6 Transcrição da voz

Objetivo	Transformar a sua fala em texto com o horário de cada palavra.
Instalar	faster-whisper e as bibliotecas do CUDA 12 (`nvidia-cublas-cu12` , `nvidia-cudnn-cu12`). Na primeira execução, o modelo `medium` (~1,5 GB) é baixado uma vez e fica em cache.

Código	• `transcricao/whisper.py`: modelo `medium` com `compute_type="int8"` na GPU, idioma `pt`, filtro de silêncio (`vad_filter`) e `word_timestamps`. • **Vocabulário**: um texto inicial (`initial_prompt`) com nomes de campeões e gírias do config, para o Whisper acertar "Yasuo", "gank", "flash" etc. O Whisper só aproveita um texto inicial curto (cerca de 224 tokens), então a lista deve ter só os termos que mais aparecem. Os mais importantes também podem ir em `hotwords`, outro parâmetro do faster-whisper. • Se faltar memória na placa, cai para o `small`. Se não tiver GPU, usa a CPU, mais devagar. • Salva no banco a cada trecho transcrito. Se for interrompido, continua do último trecho salvo. • **Versão fixada**: as versões atuais do `ctranslate2` (o motor do faster-whisper) usam CUDA 12 + cuDNN 9 (*conferido no README oficial*). O lockfile do uv trava essa versão, para uma atualização futura para CUDA 13 não quebrar a 1050 Ti. • Ao terminar, **libera a memória da GPU** antes da próxima tarefa (regra: uma tarefa de GPU por vez). • Exporta legenda `.srt` para conferência. • **Feito em 2026-10-19, com estes detalhes:** testado na GTX 1050 Ti com o driver 582.78: o `ctranslate2` 4.8.3 pede `cublas64_12.dll` (CUDA 12; um teste lê a DLL e avisa se uma atualização trocar para o CUDA 13), e o cuDNN 9.27 roda na placa. As bibliotecas da NVIDIA (~1 GB) ficam no grupo `gpu` do `pyproject.toml`: o `uv sync` as instala, e a CI, sem placa de vídeo, as pula. **Vocabulário como `hotwords`, e não `initial_prompt`**: no código do faster-whisper 1.2.1, o `initial_prompt` só vale para a primeira janela de 30 s quando `condition_on_previous_text` está desligado (o recomendado em áudios longos, para o Whisper não entrar em loop repetindo frases), enquanto os `hotwords` entram em toda janela; os dois são cortados em 223 tokens. A transcrição é feita em blocos de 10 min (`[transcricao] minutos_por_bloco`), salvos um a um no banco; fora o último, só ficam as frases que terminam 3 s antes do fim do bloco, e o bloco seguinte começa no fim da última frase aceita, para nenhuma frase sair cortada ou repetida na emenda. Entre um bloco e outro, se começar uma live, a tarefa pausa e volta para a fila sem contar como tentativa. O áudio entra no Whisper já lido (o faster-whisper 1.2.1 é incompatível com o PyAV 19 para abrir arquivos). O modelo é baixado em `dados/modelos/`, e a legenda vai para `dados/transcricoes/`. Falta de memória na placa, ao abrir ou no meio, troca para o `small`. Sem internet na primeira vez (o modelo ainda não baixado), o erro diz isso com clareza e a tarefa tenta de novo sozinha. Se o WAV da voz tiver sumido (ex.: `dados/trabalho` apagada à mão), a transcrição extrai de novo sozinha. O `medium` ocupa cerca de 1,6 GB da placa durante a transcrição (medido), além de ~1 GB que a tela já usa, e a memória volta ao normal no fim. A demo já vem com a transcrição pronta (não precisa de placa de vídeo). **Medido** em 5 gravações antigas (47 min, só com a faixa da mistura, então música e jogo também passam pelo filtro de silêncio): cerca de 3 vezes mais rápido que o tempo real (26 min em 8,5 min); só a voz, 13 vezes (91 s em 7 s). Uma live de 3 h fica entre 15 e 60 min. Nenhuma frase repetida ou sobreposta nas emendas.
Funcionalidade	Toda live ganha uma transcrição pesquisável, que alimenta o detector de fala e as legendas dos clipes.

Testes	A conversão do resultado do Whisper para o banco e para `.srt` e a retomada após interrupção são testadas com dados prontos, sem GPU, para a CI rodar. A transcrição real é testada à mão.
Pronto quando	Uma gravação real de 3h é transcrita numa madrugada, com o tempo gasto e o modelo usado registrados no log.

Etapa 7 Coletor do LoL e alinhamento de tempo

Objetivo	Registrar os eventos da partida durante a live e saber em que segundo da gravação cada um aconteceu.
Instalar	httpx. O certificado da Riot (`riotgames.pem`), disponível na documentação oficial para desenvolvedores, fica no repositório para a conexão HTTPS ser verificada.

Código	• `jogo/lol_coletor.py` : a cada 2 segundos consulta a Live Client Data API em `https://127.0.0.1:2999/liveclientdata/` : ◦ `eventdata` : lista de eventos da partida. ◦ `gamestats` : o tempo de jogo atual. ◦ `activeplayername` : o seu nome na partida. Fora de partida, a API não responde e o coletor só espera, sem encher o log de erro. • Eventos guardados: `ChampionKill` , `Multikill` , `Ace` , `FirstBlood` , `DragonKill` , `BaronKill` , `TurretKilled` . Cada um é marcado como **seu** ou não, comparando com o seu nome (abate seu, morte sua, participação). O nome vem como Riot ID (`Nome#TAG`), e os eventos podem trazer só o `Nome` , então a comparação ignora a parte do `#TAG` . • Sem duplicar: cada evento tem um `EventID` , e o coletor só salva os novos. • `jogo/alinhamento.py` : a cada consulta, guarda o par (horário do PC, tempo de jogo). Com isso calcula **quando a partida começou no relógio real**. O horário real de cada evento é esse início mais o `EventTime` ; subtraindo o início da gravação, sai o segundo exato no vídeo. • Roda dentro do serviço da etapa 4, que já abre no login. • **Feito em 2026-10-19, com estes detalhes:** seguindo a ADR 0003, o cliente HTTP ficou em `integracoes/lol.py` e o resto em `backend/jogo/` (`eventos.py` , `alinhamento.py` , `coletor.py`). Formatos conferidos no exemplo oficial da Riot (`liveclientdata_events.json`): `Stolen` vem como texto ("False"), e o `FirstBlood` traz `Recipient` . Além dos tipos previstos, guarda `HeraldKill` , `InhibKilled` e `GameEnd` (vitória ou derrota). O certificado é a autoridade "LoL Game Engineering Certificate Authority" (válida até 2043); a verificação é completa, com o nome do host, e só ela é aceita. O `EventID` recomeça a cada partida, então um evento é identificado pelo par (início da partida, `EventID`), com regra `UNIQUE` no banco (migração 0004); se o app reabrir no meio de uma partida, o coletor reaproveita a partida salva com início a até 10 s. O início da partida é a mediana das 5 amostras mais recentes de (horário do PC − tempo de jogo); leituras com o tempo de jogo zerado são ignoradas. O coletor roda numa thread própria dentro do serviço (uma tarefa da fila pode levar minutos) e liga ou desliga em `[lol] coletar` . Novos comandos: `clipador eventos` (eventos de uma gravação, com o segundo no vídeo) e `clipador coletar-lol` (só o coletor, para testar). **Testado numa partida real** (2026-10-19): abate, morte e assistência marcados pelo Riot ID `Nome#TAG` , horários de todos os eventos coerentes com o mesmo início de partida (variação de 40 ms em 4 min). Sem a duração da gravação (antes da etapa 5), o `clipador eventos` procura até 12 h depois do início e avisa, para eventos de outra live não aparecerem. Falta conferir o segundo no vídeo numa live gravada pelo OBS. Limites: pausa (só em personalizadas) e replay assistido durante a live podem deslocar os horários.
Funcionalidade	Depois da live, o banco sabe em que segundo da gravação aconteceu cada evento e quais foram seus.
Testes	**Obrigatórios**: partida que começa depois da gravação, gravação que passa da meia-noite, evento fora do intervalo da gravação, várias partidas na mesma live, evento repetido que não pode ser duplicado e um evento marcado como seu pelo nome, com e sem `#TAG` .

Pronto quando	Numa partida real, um abate seu aparece no segundo certo da gravação, com margem de 2 s.

Etapa 8 Detectores e nota

Objetivo	Achar os melhores momentos da live e dar uma nota de 0 a 100 para cada um.
Instalar	Nada novo.

Código	• `detectores/base.py` : interface `Detector.detectar(entrada) -> list[Sinal]` (no planejamento era `list[Momento]` ; mudou porque um momento junta sinais de vários detectores, ver ADR 0005). Cada detector devolve, por trecho, um sinal de 0 a 1 e pode ser ligado ou desligado no config. • `volume_voz.py` (**sinal principal**): mede o volume da voz em decibéis, em janelas de 0,5 s, e compara com a **mediana** dos 60 s anteriores (o seu volume normal). Pico = ficar pelo menos **10 dB** acima do normal (configurável) por 1 s ou mais. Gritos e risadas altas caem aqui. • `fala.py` : expressões faladas, configuráveis ("meu Deus", "não acredito", "caramba"...), e fala acelerada (palavras por segundo bem acima da sua média). Risada não entra aqui: o Whisper não escreve risada como "kkkk". • `lol.py` : pontos por evento (ex.: pentakill > barão > abate). Eventos seus valem mais que os do resto da partida. • `nota.py` : ◦ Junta sinais próximos num momento só. ◦ Define a janela do clipe: começa alguns segundos antes do primeiro sinal (padrão 10 s) e termina depois do último (padrão 5 s), com duração entre 15 e 60 s. Esses padrões depois são ajustados automaticamente pelos seus ajustes de corte (etapa 14). A janela nunca passa do começo ou do fim da gravação. ◦ Nota = 100 × média ponderada dos sinais. Os pesos iniciais ficam no config; depois das etapas 14 e 17, a versão aprovada fica na tabela `PesosNota` . ◦ Guarda só os melhores, até `max_momentos_por_live` (padrão 30), para não cortar centenas de clipes. • **Feito em 2026-10-19, com estes detalhes:** cada detector devolve **sinais** (trecho, força de 0 a 1 e motivo), e o `nota.py` os junta em momentos: um momento bom costuma juntar vários aspectos, e é a nota que pesa cada um. O registro dos plugins fica em `detectores/detectar.py` (`DETECTORES`), e cada um tem a sua seção em `[deteccao]` (ligado, peso e parâmetros). **Volume**: o "normal" é a mediana das janelas **com voz** do último minuto (abaixo de -60 dBFS é silêncio ou o noise gate e não entra), e nunca fica mais de 6 dB abaixo do seu volume típico na live, para a primeira frase depois de um tempo calado não parecer um grito; força 0,5 no limiar e 1 com o dobro do excesso. **Fala**: expressões comparadas sem acento, maiúscula e pontuação; 0,8 cada, 1 com outra a menos de 10 s; fala acelerada = palavras em 5 s comparadas com a sua mediana (1,6 vezes vale 0,5). **LoL**: tabela de força por evento (pentakill 1, quadra 0,9, triplo 0,8, barão 0,7, duplo e ace 0,6, first blood e vitória 0,5, abate, dragão e arauto 0,4, torre 0,2; roubado +0,2), vezes 100% se é seu, 80% sua morte, 70% assistência e 30% de outro jogador. **Nota**: o detector sem dados na live (ex.: LoL numa live de outro jogo) sai da média, para a nota máxima continuar sendo 100. Acrescentado: `nota_minima` (10), que descarta ruído (ex.: só a torre de um aliado). Um grupo de sinais que não cabe num clipe (ex.: 3 minutos de luta com gritos a cada 5 s) é dividido em pedaços de até 60 s, para nenhum pico se perder; só um sinal único longo demais fica com a janela ancorada nele; janelas sobrepostas viram uma se couberem em 60 s, senão são cortadas entre os sinais. Novo comando `clipador momentos` . Na demo: exatamente os 4 picos, com notas de 83 a 100. Nas 5 gravações antigas (só a faixa da mistura, sem LoL): a pipeline inteira rodou sem erro, a detecção levou menos de 1 s por gravação (uma live de 3 h: ~1 s), e o WAV é lido em blocos de 10 min (~40 MB). Falta o "pronto quando" numa live real com a voz na faixa 2.

Funcionalidade	Cada live gera uma lista de momentos ordenada por nota, com o motivo de cada nota ("grito + abate duplo seu").
Testes	**Obrigatórios**: • Cada detector acha os picos da gravação demo e ignora o ruído. • Um aumento lento de volume não conta como pico. • Momentos sobrepostos viram um só, e a janela respeita os limites de 15 e 60 s. • A nota respeita os pesos. • Uma live sem partida de LoL funciona só com voz e fala.
Pronto quando	Numa live real, pelo menos metade dos 15 primeiros momentos parecem bons para você.

Etapa 9 Cortes com FFmpeg

Objetivo	Transformar os momentos em vídeos prontos para postar.
Instalar	Nada novo. Usa o FFmpeg com NVENC.

Código	• `cortes/ffmpeg.py` : monta e roda os comandos do FFmpeg, lendo direto do MKV. ◦ **Áudio**: mistura só as faixas 2 (voz) e 3 (jogo), **sem a música**, com o volume de cada uma no config. O volume final é normalizado (filtro `loudnorm`, por volta de -14 LUFS, o padrão de YouTube e TikTok). ◦ **Vídeo**: corte com recodificação, preciso ao quadro, em `h264_nvenc` (se funcionar; senão, `libx264` no processador, ver ADR 0006). Saída em **MP4** (H.264 + AAC), o formato mais aceito pelas plataformas. • `horizontal.py` : clipe 16:9 do momento, na resolução da gravação. • `vertical_a.py` : 1080×1920. O vídeo inteiro fica no centro, e uma cópia ampliada e desfocada (`boxblur`) preenche o fundo. • `vertical_b.py` : **PAUSA**. O desenvolvimento para aqui, e essa parte é revisada com atenção antes de implementar (seção 5). • **Legendas** opcionais queimadas no vertical, ligadas ou desligadas no config. Para cada clipe, o app gera um `.srt` só com as palavras daquele trecho, com os tempos recalculados para começar do zero, e queima com o filtro `subtitles` . • **Thumbnail**: um quadro do pico do momento, usada quando um clipe horizontal sobe como vídeo normal no YouTube (etapa 15). • Quais formatos gerar fica no config. Nome dos arquivos: `data_nota_id_formato.mp4` , em `dados/clipes/` . • As mesmas funções atendem a tarefa `recortar` , que corta de novo um clipe quando você ajusta o início ou o fim na revisão (etapa 12). • **Feito em 2026-10-20 (menos o vertical B), com estes detalhes: NVENC**: o FFmpeg 9.0.2 exige a API 13.1 do NVENC (driver 610 ou mais novo), e a GTX 1050 Ti parou no driver 580 (API 13.0); por isso o encoder é escolhido sozinho (`[cortes] encoder = "auto"`): testa o NVENC uma vez e, sem ele, corta no processador com o libx264 (preset medium, CRF 21). Medido: 20 s de vertical em 1080p em ~14 s (ADR 0006). Arquivos em `cortes/` : `comandos.py` (monta os comandos, testados sem o FFmpeg), `legenda.py` e `cortar.py` (executores `cortar` e `recortar`). Áudio: voz (1,0) + jogo (0,6), sem a música, `loudnorm` numa passada (medido na demo: -14,2 LUFS); gravação sem a faixa do jogo sai só com a voz. Vertical A: o desfoque do fundo é feito em 270×480 e ampliado (mesma aparência, bem mais rápido). Legenda do clipe: linhas de até 4 palavras ou 28 letras, quebradas nas pausas, a 26% do pé da tela (mais baixo, os botões do TikTok e dos Shorts cobrem); passada ao FFmpeg pelo nome, com ele rodando na pasta da legenda, para evitar os escapes de caminho do Windows dentro do filtro. Thumbnail (JPG de 1280 de largura) no pico do momento: o `Momento` ganhou `pico_s` , o centro do sinal que mais pesou na nota (migração 0005). Retomável: cada clipe é salvo no banco assim que fica pronto, e a tarefa pausa entre clipes se começar uma live. No `recortar` , a thumbnail fica sempre dentro do novo trecho, e um trecho com menos de 1 s (ex.: fim antes do início) vira erro com mensagem clara, sem 3 tentativas. Na demo: 8 clipes e 4 thumbnails em ~1 min, corte preciso ao quadro (conferido pelo relógio da demo), sem a música (o tom do jogo 58 vezes mais forte que o da música).
Funcionalidade	De manhã, a pasta `dados/clipes/` tem os clipes do dia, nos formatos escolhidos, prontos para revisar.
Testes	Os comandos FFmpeg montados batem com o esperado (testados sem rodar o FFmpeg). Na demo, o clipe tem a duração e a resolução certas, áudio sem a faixa de música e abre como MP4 válido. A legenda do clipe começa no tempo zero e bate com a fala.

Pronto quando	Uma live real gera os clipes durante a madrugada sem erro, um formato de cada vez na GPU, e eles tocam no celular.

Etapa 10 Títulos e hashtags com IA local

Objetivo	Sugerir títulos e hashtags que pareçam escritos por você, não por um robô, para o app ficar mais autônomo.
Instalar	• **Ollama** (gratuito, roda modelos de IA no próprio PC, sem internet): `winget install Ollama.Ollama`. *Conferido na documentação oficial:* suporta a GTX 1050 Ti, mas exige **driver NVIDIA 570 ou mais novo** para essa placa (na prática, entre 570 e 580; veja a etapa 0). • Modelo **qwen3:4b** (2,5 GB, cabe nos 4 GB da placa; atende mais de 100 idiomas, incluindo português): `ollama pull qwen3:4b`. Se faltar memória, cai para o `qwen3:1.7b` (1,4 GB). O modelo fica no config, para trocar por um melhor no futuro. • Biblioteca Python `ollama`, via uv.

Código

- `titulos/gerador.py` : para cada momento, envia ao modelo o trecho da transcrição (o que você falou), os eventos do jogo (campeão, pentakill...) e as instruções de estilo. Recebe **3 sugestões de título** e as hashtags, numa resposta em JSON (o Ollama permite exigir esse formato).
- **Estilo**: frases curtas, com o seu jeito de falar, curiosidade ou exagero, sem cara de "melhores momentos #12". Exemplos do estilo esperado:
  - Pentakill de Yasuo com grito → "*EU NÃO ACREDITO NO QUE ESSE YASUO FEZ*", "*O time desistiu. O Yasuo não.*"
  - Flash errado com risada → "*Eu juro que apertei o flash*", "*O flash mais triste da história do LoL*"
- **Aprende o seu estilo**: os últimos títulos que você escolheu ou editou na revisão (tabela `Decisao` ; até uns 10, para caber no pedido) entram como exemplos nas próximas gerações. Quanto mais você usa, mais os títulos ficam com a sua cara.
- **Tipo de cada título**: o próprio modelo marca cada sugestão como "frase sua", "pergunta", "exagero" ou "descrição". Esse tipo é usado na etapa 17 para descobrir quais tipos dão mais engajamento.
- **Hashtags**: mistura a sua lista fixa do config (ex.: `#leagueoflegends #lolbr`) com as que o modelo sugere a partir do momento (ex.: `#yasuo #pentakill`). A partir da etapa 17, a escolha também considera quais hashtags trazem mais engajamento. Padrão: 3 a 5 hashtags por vídeo. Regras do YouTube, *conferidas na ajuda oficial*: ele mostra no título até 3 das hashtags que considera mais interessantes; com mais de 60, todas são ignoradas; hashtags sem relação com o vídeo podem levar à remoção dele. Por isso o app só sugere hashtags ligadas ao conteúdo do clipe (jogo, campeão, o que aconteceu).
- **Limites de texto**: título do YouTube até 100 caracteres; legenda do TikTok até 2.200 caracteres (*conferido na documentação da Content Posting API*). O app corta ou avisa antes de passar.
- Roda na fila depois dos cortes. Usa a GPU, então segue a regra de uma tarefa de GPU por vez, e descarrega o modelo da memória ao terminar (`keep_alive=0`). Pede o modo sem "raciocínio" do modelo, que é mais rápido para textos curtos.
- Se o Ollama não estiver instalado ou falhar, o app usa um título simples a partir da frase mais marcante da transcrição, e nada trava.
- **Feito em 2026-10-21, com estes detalhes:** Ollama 0.40.7 instalado pelo winget (abre sozinho no login pelo atalho na pasta Inicializar) e biblioteca `ollama` 0.6.3. Na 1050 Ti, o Ollama usa as bibliotecas CUDA 12 dele (as de CUDA 13 pulam a placa, conferido no log). Arquivos em `titulos/` : `gerador.py` (pedido, JSON Schema no `format` , validação, plano B), `texto.py` (limite de 100 caracteres, hashtags, frase marcante) e `gerar.py` (executor, último da cadeia). O modelo fica na placa durante a tarefa (`keep_alive` de 10 min, para não recarregar a cada momento) e é descarregado no fim (`keep_alive=0`), mesmo se a IA falhar no meio. Ajustes medidos com o modelo real: o momento vai descrito **na voz do streamer** ("eu gritei", "barão roubado (fui eu)"), porque, escrito para a revisão ("pentakill seu", "+24 dB") ou na 3ª pessoa ("o streamer gritou"), o modelo copiava isso para os títulos; o tipo repetido no fim do título ("(frase_sua)") é limpo; hashtags com palavras do próprio pedido ("#loleventos") são descartadas. O pedido leva só os 5 eventos de jogo mais fortes do trecho (numa luta longa podem ser dezenas, e um modelo pequeno se perde). Plano B: a frase dita mais perto do pico e o evento de jogo mais forte em 1ª pessoa ("Barão roubado", "Morri"). Medido na 1050 Ti: ~25 s por momento (o 1º, ~55 s, carrega o modelo; 3,5 GB na placa durante a tarefa, volta a ~1,1 GB no fim). Na demo, os títulos são os

	prontos do roteiro (`[titulos] fonte = "demo"`), para ela rodar sem o Ollama. O `clipador momentos` mostra o 1º título e as hashtags de cada momento.
Funcionalidade	Cada clipe chega na revisão com 3 títulos e hashtags prontos. Você só escolhe ou ajusta.
Testes	Com o modelo simulado: o pedido leva a transcrição e os eventos certos, a resposta JSON é validada, títulos acima do limite são cortados (YouTube: 100 caracteres) e a falha do Ollama cai no título simples. A qualidade dos títulos é avaliada à mão, com lives reais.
Pronto quando	Numa live real, para a maioria dos clipes, pelo menos uma das 3 sugestões serve sem edição ou com edição pequena.

Etapa 11 Desligamento automático

Objetivo	Desligar o PC sozinho depois do processamento, sem atrapalhar se você estiver usando.
Instalar	Nada novo. Usa funções do próprio Windows.
Código	• `sistema/ociosidade.py` : `GetLastInputInfo` diz há quanto tempo o teclado e o mouse estão parados. Isso só funciona dentro da sua sessão do Windows, e é por isso que o app abre pela chave "Run" no login, e não como serviço do sistema. • `sistema/desligamento.py` : quando a fila acaba, verifica a cada minuto se você está ocioso há 10 min. Programas abertos não impedem. ◦ Gravação em andamento conta como fila. ◦ **Tarefas agendadas para o futuro não contam** (ex.: coleta de métricas daqui a 7 dias, reenvio depois que a cota zerar). Se contassem, o PC nunca desligaria. • Depois, contagem de 60 s com `shutdown /s /t 60` ; o Windows mostra um aviso na tela. Mexer no mouse ou no teclado faz o app rodar `shutdown /a` e voltar a esperar. Você também pode digitar `shutdown /a` . • **Atenção**: assistir a um vídeo sem tocar no PC conta como ocioso. Nesse caso o aviso de 60 s aparece, e basta mexer o mouse. • Config: `desligar_ao_terminar` , `minutos_ocioso = 10` e `modo_teste` , que só escreve "desligaria agora" no log. O `modo_teste` vem **ligado** por padrão.
Funcionalidade	Você vai dormir. O app termina tudo e desliga o PC.
Testes	A regra é testada com relógio e ociosidade simulados, sem desligar de verdade: fila pendente, gravação em andamento, só tarefas agendadas para o futuro (deve desligar), usuário ativo, cancelamento durante a contagem e modo de teste.
Pronto quando	Uma noite inteira em modo de teste mostra no log o horário certo em que o PC teria desligado. Só então você desliga o `modo_teste` .

Fase C · Revisão o que você usa no dia seguinte

Etapa 12 API de revisão (backend)

Objetivo	Expor clipes e decisões para a interface web.
Instalar	FastAPI e Uvicorn.

Código	• `web/api.py`, ouvindo só em `127.0.0.1` (não fica visível na rede): ◦ `GET /api/momentos`: momentos do dia ordenados por nota, com os formatos já cortados e os títulos e hashtags sugeridos de cada um. ◦ `GET /api/clipes/{id}/video`: envia o vídeo com suporte a *Range*, para dar para avançar e voltar no player. ◦ `POST /api/momentos/{id}/decisao`: aprovar ou recusar, com os formatos e destinos escolhidos, o título e as hashtags. ◦ `DELETE /api/momentos/{id}/decisao`: desfazer uma decisão (antes da publicação). ◦ `POST /api/momentos/{id}/ajuste`: novo início e fim do corte. Cria uma tarefa que corta de novo só esse clipe. Se o clipe já foi aprovado, a publicação espera o novo corte ficar pronto. ◦ `GET /api/status`: fila, erros e cota do YouTube. ◦ `GET /api/estatisticas`: dados dos gráficos. • **Limite diário** (padrão 15): a revisão mostra os 15 momentos de maior nota do dia; o restante fica num "ver mais". • **Aprovar** = escolher formatos e destinos. Os destinos padrão ficam no config (ex.: vertical → Shorts e TikTok). Os 3 títulos e as hashtags sugeridos na etapa 10 aparecem para você escolher ou editar. • CLI: `clipador web` sobe a API e abre o navegador.
Funcionalidade	A documentação interativa em `/docs` já permite revisar, mesmo sem a interface pronta.
Testes	Cada rota com o banco demo: ordenação, limite diário, decisão salva com formatos, destinos, título e hashtags, desfazer, ajuste do corte cria a tarefa de novo corte, momento inexistente devolve 404, vídeo devolve 206 quando o pedido usa *Range*.
Pronto quando	Dá para revisar a demo inteira pelo `/docs`.

Etapa 13 Interface de revisão (frontend)

Objetivo	Revisar os clipes do dia rápido, quase só pelo teclado.
Instalar	Via npm: React, Vite, TypeScript, Tailwind, Recharts (gráficos) e Vitest.

Código	• `frontend/` com 3 telas: ◦ **Revisão**: player grande com autoplay, nota, motivo, título editável e formatos. ◦ **Fila e status**: o que está processando, o que deu erro e a cota do dia. ◦ **Estatísticas**: gráficos de precisão e de sinais; engajamento a partir da etapa 17. • Atalhos: ◦ **A** aprova e **R** recusa. ◦ **U** desfaz. ◦ ← → navega e **espaço** pausa. ◦ **1/2/3** troca o formato exibido. ◦ **T** edita o título e **Tab** alterna entre as 3 sugestões. ◦ **[** e **]** recuam ou avançam o início do corte em 1 s; **Shift + [** e **Shift +]** fazem o mesmo com o fim. • **Ajuste do corte**: uma linha do tempo embaixo do player, com alças de início e fim para arrastar. O ajuste é mostrado na hora no player; ao aprovar, o app corta de novo só esse clipe. • Enquanto você digita no campo de título, os atalhos ficam desligados (senão, digitar "a" aprovaria o clipe). **Esc** sai do campo. • Contador "7 de 15 revisados hoje". • **Autoplay**: o navegador só toca vídeo com som depois de você interagir com a página. O primeiro clipe espera uma tecla; os seguintes tocam sozinhos. • No desenvolvimento, o Vite repassa as chamadas `/api` para o FastAPI (proxy). No uso normal, o build vira arquivos estáticos servidos pelo próprio FastAPI, e o Node não é necessário.
Funcionalidade	Revisar 15 clipes leva poucos minutos. Toda decisão fica salva.
Testes	Vitest nos atalhos, no desfazer, no contador e no ajuste do corte (início nunca depois do fim, limites de 15 a 60 s). Teste manual completo com a demo.
Pronto quando	Você revisa uma live real inteira sem tocar no mouse. Um GIF dessa tela vai para o README.

Etapa 14 Aprendizado com as aprovações

Objetivo	Usar as suas decisões para medir e melhorar o detector. É o diferencial de portfólio.
Instalar	scikit-learn.

Código	• `aprendizado/metricas.py`: precisão por dia ("dos 15 sugeridos, aprovei 9") e quais sinais aparecem mais nos aprovados. • `aprendizado/pesos.py`: com 100 decisões ou mais, treina uma regressão logística. As entradas são os sinais de cada momento; a saída é "aprovou?". Os coeficientes, normalizados, viram os pesos sugeridos. • **Comparação justa**: a precisão "antes e depois" é medida com validação cruzada, em decisões que o modelo não viu no treino. Você vê os números e aceita ou recusa. A versão aceita vai para a tabela `PesosNota`. • **Limitação**, documentada no README: só os momentos que você revisou têm decisão. O modelo aprende entre os já bem avaliados, não sobre a live inteira. • Os gráficos aparecem na tela de Estatísticas. • **Janela do corte**: a média dos seus ajustes de início e fim (etapa 13) atualiza os padrões da etapa 8. Se você sempre adianta o início em 3 s, os próximos cortes já começam 3 s antes. • Na etapa 17, o mesmo mecanismo passa a usar também o engajamento do público.
Funcionalidade	Com o tempo, a nota fica mais parecida com o seu gosto.
Testes	Métricas com decisões inventadas de resultado conhecido. Um sinal que sempre leva à aprovação recebe peso maior. O treino se recusa a rodar com menos de 100 decisões.
Pronto quando	O README mostra um gráfico real da precisão ao longo do tempo. ADR: "Por que regressão logística".

Fase D · Publicação só o que você aprovou

Etapa 15 Upload no YouTube

Objetivo	Enviar os clipes aprovados para o YouTube, respeitando a cota.
Instalar	google-api-python-client e google-auth-oauthlib.
Configurar (você)	• No Google Cloud Console: criar um projeto e ativar a YouTube Data API v3. • Tela de consentimento OAuth: tipo "Externo", permissão `youtube.upload`, publicada "Em produção" (em "Teste" o login expira em 7 dias). O Google mostra um aviso de "app não verificado" no login; para uso pessoal, basta continuar. • Criar uma credencial OAuth do tipo "App para computador" e salvar o `client_secret.json` fora do git. • **Cada passo que envolve o Google é combinado antes de ser feito.**

Código	• `publicacao/youtube.py`: login uma vez (o token fica salvo fora do git e é renovado sozinho). Upload retomável, que continua de onde parou se a internet cair. O ID do vídeo devolvido pelo YouTube fica salvo em `Publicacao`, para as métricas da etapa 17. • Título e hashtags que você aprovou (as hashtags vão na descrição), mais descrição e tags a partir de modelos no config. Categoria "Jogos" (`categoryId` 20) e **"não é conteúdo para crianças"** (`selfDeclaredMadeForKids = false`), declaração que o YouTube exige. • Shorts = vertical de até 3 min. Clipes horizontais sobem como vídeo normal. • **Avisar inscritos** (`notifySubscribers`) fica configurável. Muitos Shorts por dia podem gerar notificação demais para quem é inscrito. • Thumbnail personalizada só nos clipes horizontais: exige canal verificado por telefone e gasta ~50 unidades. Nos Shorts, o YouTube escolhe o quadro. • `publicacao/cota.py`: conta os envios do dia (limite de 100) e as unidades das outras ações (limite de 10.000; thumbnail = 50). Se passar do limite, para e continua depois que a cota zera (4h–5h em Brasília). • **Agendamento automático**: o vídeo sobe como privado com data e hora de publicação (`publishAt`), e o YouTube publica sozinho no horário. Os horários padrão ficam no config, com um intervalo mínimo entre publicações para não soltar tudo de uma vez.
Funcionalidade	Você aprova e o app envia. O link fica salvo no banco e aparece na interface.
Testes	Cota, fila e montagem do pedido (título, categoria, privacidade, declaração para crianças, aviso aos inscritos, horário de publicação respeitando o intervalo mínimo) testados com a API simulada, sem gastar cota. Um upload real privado é testado à mão.
Pronto quando	Um Short aprovado aparece agendado no seu canal, com título, descrição e categoria certos, e é publicado sozinho no horário.

Etapa 16 TikTok

Objetivo	Levar os verticais aprovados ao TikTok, do jeito mais simples para o mais automático.
Instalar	Nada novo (httpx).

Código	• **(a) Pasta**: copia o vertical aprovado para `dados/exportar/tiktok/` com a legenda e as hashtags aprovadas num `.txt` ao lado. Funciona desde o primeiro dia. • **(b) Rascunho**: criar um app no TikTok for Developers com a Content Posting API, testar no sandbox e fazer login OAuth com a permissão `video.upload`. O app envia o vídeo pelo endpoint de upload para a caixa de entrada (`/v2/post/publish/inbox/video/init/`), e você finaliza no celular. Para usar fora do sandbox, o app do TikTok precisa ser aprovado na análise. ◦ Para mandar o app para análise, o TikTok pede ícone, descrição e links de **Termos de Uso** e **Política de Privacidade**. Dá para hospedar essas páginas de graça no GitHub Pages. ◦ Login como **app de desktop**, *conferido na documentação oficial*: o endereço de retorno (*redirect URI*) pode ser `http://127.0.0.1:{porta}/callback/` (só `localhost` ou `127.0.0.1`, com porta) e precisa estar cadastrado no app. O login exige **PKCE** (um código de verificação gerado a cada login, com desafio em SHA-256 codificado em hexadecimal). ◦ **Login que dura** (*conferido na documentação oficial*): o token de acesso vale 24 h e o de renovação (*refresh token*) vale 365 dias. O app renova o acesso sozinho e avisa quando faltar pouco para o login vencer. ◦ **Nenhum pedido de aprovação ao TikTok é feito sem ser combinado antes.** • **(c) Post direto** (permissão `video.publish`): só depois da auditoria do TikTok. Antes dela, os posts ficam privados. Antes de cada post, o app consulta as informações do criador (`creator_info`): opções de privacidade permitidas e duração máxima. Isso a documentação exige.
Funcionalidade	Os aprovados chegam ao TikTok com o mínimo de trabalho manual que a plataforma permite.
Testes	Exportação para a pasta (vídeo e `.txt`). Montagem dos pedidos com a API simulada.
Pronto quando	A opção (a) funciona sempre. A (b) funciona no sandbox.

Etapa 17 Métricas de engajamento

Objetivo	Medir como cada corte publicado foi no YouTube e no TikTok e usar isso para a nota prever o que o **público** engaja, não só o que você aprova.
Instalar	Nada novo. Usa as mesmas bibliotecas das etapas 15 e 16.
Configurar (você)	• Google: ativar a **YouTube Analytics API** no mesmo projeto e incluir as permissões de leitura `youtube.readonly` (estatísticas dos seus vídeos) e `yt-analytics.readonly` (retenção). A `youtube.upload` da etapa 15 serve só para enviar. O app pede o login de novo uma vez. • TikTok: adicionar o produto **Display API** ao app, com a permissão `video.list`. • **Cada passo que envolve Google ou TikTok é combinado antes de ser feito.**

Código	• `engajamento/coleta.py`: para cada publicação, coleta as métricas três vezes, **24 h, 72 h e 7 dias** depois. São tarefas agendadas na própria fila. Fontes, *conferidas na documentação oficial*: ◦ **YouTube Data API** (`videos.list`, parte `statistics`): visualizações, curtidas e comentários. Custa 1 unidade por chamada, e uma chamada pode trazer vários vídeos de uma vez (passando vários IDs). ◦ **YouTube Analytics API**: `averageViewDuration` e `averageViewPercentage`, quanto do vídeo as pessoas assistem. É o sinal mais importante nos Shorts. Os dados do Analytics costumam chegar com atraso de alguns dias, por isso a retenção conta só nas coletas de 72 h e 7 dias. ◦ **TikTok Display API** (`video.list`): `view_count`, `like_count`, `comment_count` e `share_count`. A lista traz só os vídeos **públicos**, do mais novo para o mais antigo, até 20 por página. Vídeos privados (como os de post direto antes da auditoria) não aparecem. • **Ligar o rascunho do TikTok ao vídeo publicado**: no modo rascunho você finaliza a postagem no celular, então o app não recebe o ID do vídeo. Ele procura nos seus vídeos pelo horário de criação e pelo título. • `engajamento/indice.py`: transforma as métricas num **índice de engajamento** comparável entre vídeos. Ele junta três medidas, com fórmula e pesos no config: ◦ visualizações divididas pela mediana do canal naquela rede (assim o crescimento do canal não distorce a comparação); ◦ taxa de interação: curtidas + comentários + compartilhamentos, divididos pelas visualizações; ◦ retenção. • `aprendizado/engajamento.py`: com **50 cortes ou mais** com a métrica de 7 dias, treina uma regressão logística ("ficou acima da mediana?") a partir dos sinais de cada momento. A **nota final** vira uma mistura configurável, por exemplo 50% seu gosto (etapa 14) e 50% engajamento previsto. A comparação justa e o seu aceite seguem a mesma regra da etapa 14, e a versão aceita vai para `PesosNota`. • `engajamento/hashtags.py`, **hashtags que funcionam**: cada hashtag usada fica ligada às métricas do vídeo. O app compara o engajamento médio dos vídeos com e sem ela, e só conclui algo depois de a hashtag aparecer em pelo menos 5 vídeos. Para não usar só as mesmas de sempre, de vez em quando ele troca uma hashtag por outra ainda pouco testada, **sempre entre hashtags relacionadas ao clipe**: hashtag sem relação com o vídeo pode fazer o YouTube removê-lo. Esse equilíbrio entre "aproveitar o que funciona" e "testar o novo" é o problema do *multi-armed bandit*, resolvido com a técnica *epsilon-greedy*. As melhores hashtags passam a ser priorizadas nas sugestões da etapa 10. • **Títulos que funcionam**: o mesmo vale para o tipo de título (frase sua, pergunta, exagero), marcado pelo próprio modelo da etapa 10. Os tipos que dão mais engajamento passam a ser pedidos com mais frequência. • A tela de Estatísticas ganha: ranking dos cortes por engajamento, ranking das hashtags, engajamento por tipo de momento (grito, abate, fala) e comparação entre YouTube e TikTok.
Funcionalidade	Você vê quais cortes funcionaram com o público, e a nota passa a priorizar os tipos de momento que geram mais engajamento.

Testes	• Índice calculado com números inventados de resultado conhecido. • Coletas de 24 h, 72 h e 7 dias agendadas certo. • Rascunho do TikTok ligado ao vídeo certo pelo horário. Se o vídeo ainda não aparece (não foi finalizado ou está privado), o app tenta de novo na próxima coleta, sem erro. • Ranking de hashtags com dados inventados; uma hashtag com menos de 5 usos não é ranqueada; a exploração respeita a taxa configurada. • Treino se recusa a rodar com menos de 50 cortes. • APIs simuladas, sem gastar cota.
Limitações	Engajamento também depende de horário, tendências e do algoritmo de cada rede. Com poucos vídeos, o modelo erra bastante. Isso fica documentado no README; a etapa 19 ("melhores horários") ataca parte do problema.
Pronto quando	A tela mostra métricas reais de pelo menos 10 cortes publicados. Ao chegar em 50, sai a primeira sugestão de pesos, com a comparação antes e depois.

Fase E · Vitrine e futuro

Etapa 18 Vitrine final (versão 1.0)

Objetivo	Deixar o projeto pronto para mostrar a recrutadores.
Instalar	pytest-cov (mede quanto do código os testes cobrem).
O que fazer	• **README completo**: ◦ GIF da revisão e diagrama da pipeline em Mermaid. ◦ Gráfico de engajamento: quais tipos de momento o público mais assiste. ◦ Selos da CI e da cobertura de testes. ◦ Gráfico de precisão e limitações conhecidas. ◦ "Como rodar a demo em 3 comandos". ◦ Decisões técnicas, com links para os ADRs. ◦ Guia passo a passo para configurar as contas do Google e do TikTok (em `docs/`), para outra pessoa conseguir usar o app com os próprios canais. • **Vídeo curto** (1–2 min) mostrando o fluxo inteiro, para o README e o LinkedIn. • **Roteiro de demonstração** de 5 minutos para entrevista, usando `clipador demo`. • Revisão geral: código morto, nomes e cobertura de pelo menos 80% nos detectores e no alinhamento. • Tag `v1.0` e release no GitHub, com a lista do que o app faz.
Pronto quando	Uma pessoa que nunca viu o projeto consegue rodar a demo seguindo só o README.

Etapa 19 Futuro

Ideias

Cada ideia é um plugin ou módulo novo, sem mexer no resto, e passa pelo mesmo ciclo (plano → código → testes → aprovação):

- **Chat da Twitch** (twitchio): picos de mensagens por minuto viram um sinal.
- **Melhores horários**: YouTube Analytics API (gratuita) para descobrir quando seu público assiste e ajustar o agendamento.
- **Reação na câmera** (MediaPipe): expressões fortes viram um sinal.
- **Outros jogos**: um plugin de detector por jogo.
- **Vídeo longo** (ex.: compilado da semana): adiado de propósito. Primeiro o app fica redondo com os cortes.
- **Outras redes** (Instagram, Kwai...): por enquanto só YouTube e TikTok, na fase de testes.
- **App web no celular**: a mesma interface de revisão adaptada ao celular, com atalho na tela inicial (PWA), notificações pelo app gratuito ntfy ("15 clipes prontos", "erro na madrugada") e acesso fora de casa pelo Tailscale. Ponto a resolver antes: o PC desliga depois do processamento, então o celular só alcança o app com o PC ligado.
- **Métricas por hora de live**: cruzar o engajamento com o ponto da live em que cada momento aconteceu (1ª, 2ª, 3ª hora), com o horário e o dia da semana. Serve para decisões sobre as próprias lives, como duração ideal e o que fazer em cada parte. Só depois de alguns meses de dados, para ter parâmetro.

4. Riscos e cuidados

YouTube: cota diária. *Conferido na documentação oficial em 14/10/2026.* O upload tem uma cota própria: **100 envios por dia**, e cada envio gasta 1. As outras ações dividem 10.000 unidades por dia: ler dados custa ~1, trocar a thumbnail 50, editar um vídeo 50. As cotas zeram à meia-noite do horário do Pacífico, entre 4h e 5h em Brasília. Na prática, a cota não limita o projeto. Mesmo assim, o app conta o uso e, se passar do limite, guarda o resto na fila para o dia seguinte.

YouTube: regra antiga sobre vídeos privados. Antigamente, vídeos enviados por projetos não verificados ficavam presos como privados. *A documentação atual do* `videos.insert` *diz que eles não são mais restritos.* Com isso, o agendamento automático (`publishAt`) funciona desde o início, sem auditoria. Confira no primeiro upload real: se o Google voltar a restringir, o app sobe como privado e você publica pelo YouTube Studio.

YouTube: login que expira. Se o app OAuth do Google ficar em modo "Teste", o login (refresh token) expira a cada 7 dias e o upload da madrugada falha (*conferido na documentação OAuth do Google*). Um login sem uso por 6 meses também expira. Mude o app para "Em produção". Para uso pessoal, o Google só mostra um aviso de "app não verificado" no login.

TikTok: auditoria. Sem auditoria, a API só posta em modo privado (*conferido na documentação oficial*). A opção "salvar numa pasta" funciona desde o primeiro dia, sem depender de ninguém.

GTX 1050 Ti (arquitetura Pascal). O CUDA 13 não suporta mais placas Pascal, e os drivers mais novos da NVIDIA também estão deixando de suportar. Fique no **CUDA 12** e num driver compatível; não atualize o driver às cegas. O Ollama (etapa 10) exige driver 570 ou mais novo nessa placa, e a série 580 é a última com suporte a Pascal. Ou seja: use um driver entre **570 e 580**. *Conferido nas notas oficiais do CUDA 13.*

GPU com 4 GB. Whisper e FFmpeg disputariam a mesma memória de vídeo (hoje o FFmpeg corta no processador, porque o NVENC não funciona com o driver 580 e o FFmpeg 9: ADR 0006). Regra: uma tarefa de GPU por vez. Se o `medium` não couber, o app cai automaticamente para o `small` . O modo int8 funciona porque a 1050 Ti tem "compute capability" 6.1, o mínimo exigido pelo CTranslate2 (o motor por trás do faster-whisper) para int8 na GPU.

Direitos autorais. A faixa de música fica fora dos clipes por isso. O áudio e as imagens do LoL podem ser usados em vídeos, inclusive monetizados, pela política de conteúdo da Riot ("Legal Jibber Jabber"). Músicas de terceiros que vazarem para o microfone ou para a faixa do jogo ainda podem gerar reivindicação no YouTube.

Alinhamento de tempo. A partida começa minutos depois da gravação, e o coletor do LoL usa o tempo de jogo. Os dois precisam ser convertidos para o relógio real e alinhados. Por isso essa parte é obrigatória nos testes.

PC suspenso de madrugada. Se o Windows suspender o PC no meio do processamento, nada termina. O app impede a suspensão enquanto houver fila (SetThreadExecutionState) e libera quando acabar.

Espaço em disco. Gravando a 6.000 kbps, 3h de live ocupam cerca de 8 GB, ou uns 240 GB por mês. Defina quantos dias a gravação original fica guardada depois que os clipes forem aprovados.

5. Pendências

Vertical B (zoom na ação). Quando o projeto chegar na etapa 9, o desenvolvimento para e essa parte é revisada com atenção antes de implementar. Ponto de partida sugerido: recorte fixo configurável no centro da tela, já que no LoL o campeão fica no centro com a câmera travada.

Nome do projeto. "Clipador de lives" é provisório. O nome final entra no README e no repositório do GitHub.

6. Decisões já tomadas

- **Sinal principal**: picos de volume da sua voz (faixa do microfone). Fala e eventos do LoL entram como sinais complementares.
- **GPU**: faster-whisper `medium` int8 só na faixa da voz, com filtro de silêncio, e FFmpeg com NVENC se funcionar (com a 1050 Ti no driver 580 e o FFmpeg 9, corta no processador: ADR 0006), uma tarefa de GPU por vez.
- **Frontend**: React + TypeScript + Vite + Tailwind.
- **Escopo**: projeto completo, etapas 0 a 19 do roteiro, uma de cada vez.
- **Redes**: só YouTube e TikTok nesta fase de testes.
- **Engajamento**: as métricas dos cortes publicados também ensinam a nota (etapa 17).
- **Vídeo longo**: adiado. Fica na etapa 19 (futuro).
- **Títulos e hashtags**: gerados por IA local (etapa 10), aprendendo com as suas edições e com o engajamento.
- **Ajuste do corte** na revisão, com aprendizado da janela padrão.
- **Backup diário** do banco, guardando 14 cópias.
- **Adiados**: app web no celular com notificações e métricas por hora de live (etapa 19). O desligamento segue a regra atual.
- **Desligamento**: só quando a fila terminar *e* teclado e mouse estiverem parados há 10 min. Programas abertos não atrapalham. Contagem de 60 s cancelável e modo de teste que só escreve no log.

Fonte deste PDF: `docs/stack.html`.